ARC PRIZE 2026 · PAPER TRACK · PREPRINT

Fork Variance Is a Free Sample: Auditing a Hybrid ARC-AGI-2 Submission

Forty-four graded hidden-set draws — 43 on one public-notebook fork line, 1 on a second fork (NVARC+) — spread 26.39–31.39 (σ = 1.09): a 5.00-point spread that prices the bronze lottery. A held-out selection bake-off finds +1.67 points; a calibrated simulation projects +2.27 more; and a zero-quota real-pool benchmark refutes it (+0.00), relocating the binding constraint from selection to candidate-pool quality. Includes a free-draw audit protocol, crash-isolation and dependency-pinning lessons, and an ARC-AGI-3 replication.

defiaudit

Kaggle · ARC Prize 2026 (ARC-AGI-2 and ARC-AGI-3 tracks)
Best semi-private score 31.39 · every score anchored to a submission ID

Preprint · November 2026

Scores: Kaggle semi-private leaderboard

Abstract

We present a measurement-first audit of a mid-pack ARC-AGI-2 submission. Our entry is a two-candidate hybrid: a fork of a public leaderboard notebook and a zero-learning DSL solver that scored 0. Rather than treating the submission as a single number, we reran our fork-based entry on the identical hidden evaluation set across 45 submissions over eight weeks (44 graded draws: 43 on the fork line, one pooled draw on a second fork, NVARC+ — Section 2), logging every draw while the code moved through logged versions and three late engineering chapters. The 44 graded draws span 26.39–31.39, a 5.00-point spread ($\mu = 29.16$, sample $\sigma = 1.09$): for a benchmark graded to two decimals, the largest source of variance in our fork-cluster entry is the rerun, not the model. We show that leaderboard point estimates are order statistics, give a five-step zero-cost protocol (the *free-draw audit*) that prices resampling against engineering, and compare the fitted distribution with two related descriptive summaries. We then add a second, causal layer: a 10-selector bake-off on 37 held-out evaluation tasks (winner +1.67 points over the production default), a calibrated simulation that projected a further +2.27 from an executive verifier, and a no-quota real-pool benchmark on 40 evaluation tasks that *refuted* the simulation (+0.00) and showed forced attempt-2 diversity is harmful (−2.00). The joint lesson: once candidate pools contain a correct program, selection saturates, and the binding constraint is pool quality. Finally, we document two reliability findings from the campaign: per-task crash isolation and a third-party model privatization that permanently broke our ARC-AGI-3 reruns. Every graded score is anchored to a Kaggle submission ID; which artifacts remain public, private, or merged, is inventoried in Section 10.

1 What we submitted

ARC-AGI-2 [2] is a static abstraction benchmark: each task shows input–output grid pairs, and a solution must produce the output grid for two hidden test inputs, with two attempts allowed per task. The Kaggle competition [3] grades the best attempt, reruns every submission on a hidden set, and selects each team's best submission. Our entry has two candidates. Candidate 1 is a fork of the top public notebook in our reachable band (`mikelou1/arc-agi2-lb33-89-minimal-perfpatch`, public; its slug and title self-label it "lb33-89" — a fork-time score we cannot independently verify, the page now shows 32.22 — and the September 6 snapshot puts the top-1% cutoff at 33.06). Our fork of it lived at `defiaudit/arc-agi2-lb33-89-minimal-perfpatch`; it was later renamed and set private, its 32-version history intact (Section 10). The fork chains an LLM proposal step with test-time fine-tuning and a deterministic search layer, executed on Kaggle's 4×L4 environment under a 12-hour wall clock. Candidate 2 is a self-built DSL exact-fit solver (`defiaudit/agi2-dsl-v214r-solver`), a pure-Python chain with no learned component, whose 30-file dependency closure ships as a public dataset.

Our best score is 31.39 on the semi-private leaderboard—0.55 below the 31.94 bronze cutoff at the November 2 close (rank 669 of 3,571 then; settled once post-deadline grading drained: 715 of 3,627, cutoff 32.22)—and 2.50 below the 33.89 self-labeled in the source notebook's slug and title (not independently verifiable; the page shows 32.22; its author finished the season at 69.72, rank 33 of 3,627 at our November 5 pull). This paper is therefore not a leaderboard record. It is an audit with two layers: a statistical layer (what a rerun is worth) and a causal layer (what actually binds the score). Along the way the entry itself evolved through three engineered stages—a verbatim control (v26), a re-ranked selection layer (v29), and a crash-hardened build (v31-HARD)—each stage logged draw by draw in Section 2.

2 The draws table

Every run used the same environment and the same hidden test set; each draw carries a code-version label (39 from the export's fire descriptions, 5 from ledger notes). Across 45 submissions

since September 7 we logged 44 graded draws and one submission that returned no grade (a whole-run crash, excluded from every statistic). Numbering throughout is canonical: graded draws 1–44 in true fire order, each mapped to its submission ID in Appendix A. Fire-time descriptions carried ad-hoc labels (three were reused, one was skipped after a lost slot); the raw submission list (45 refs with dates, descriptions, and scores) ships as a public CSV alongside this paper and preserves them verbatim (Table 2).

Table 1: Draw statistics over the 44 graded draws (one further submission returned no grade and is excluded; campaign closed November 2, 2026). σ is the sample standard deviation; the population value rounds to 1.08.

n	mean	median	σ	IQR	range	spread
44	29.16	29.31	1.09	28.47–30.14	26.39–31.39	5.00

Table 2: Code-version chapters of the campaign. Later chapters add engineering; the draws distribution does not care.

Chapter	Draws	Notes
perfpatch fork line (kernels v1–v15)	1–3, 5–38	daily variance draws; seeds unfixed (the v4 seed patch survives only as a ledger note)
NVARC+ v3 (second fork)	4	one pooled two-pass draw (passA + seeded pass); verbatim fire description preserved
v26 verbatim	39	rerun self-described at fire time as byte-identical to saved kernel v12 (ref 57641015; fire description "=v12 bytes"); one further v26 submission returned no grade (ref 57646158)
v29 re-rank	40	selection battery adopted (Section 6)
v31-HARD	41–44	per-task crash isolation; zero crashes; the October 31 slot was lost to an automation fault

Draw 4 is the campaign's one submission from outside the main fork line; its verbatim fire-time description ("NVARC+ two half-cost passes pooled (passA LB-proven + seedB)") is preserved in the public submission export (Section 10). Three findings from the table. First, **unfixed draws are a random variable**: the same fork family produced 26.39 and 31.39, and the 5.00-point spread contains most of the published fork-cluster band (similar forks cluster near 28.06 on the public leaderboard; the source notebook self-labels 33.89 in its slug and title (no independent record; page shows 32.22), and its author finished the season at 69.72). The interquartile range alone spans 28.47–30.14, so most leaderboard comparisons inside the fork band are noise. Second, **the two draws the record marks as seed-controlled landed on exactly 30.14**: ref 56125103 (our ledger's seed-patched-v4 note — the export carries no fire description for it) and ref 56148554 (whose verbatim fire description names a seeded pass, "passA LB-proven + seedB"), while three runs of unpatched v1 scattered 28.89–30.97 (refs 56075910, 56100667, 56168010). The pattern is consistent with sampling RNG, not hardware scheduling, dominating the variance; the seed labels are bookkeeping entries rather than platform-verified states, and $n = 2$ pairs cannot establish the mechanism (Section 11). Third, σ **barely moves across engineering chapters**: adding a selection battery and crash hardening shifted σ only from 1.14 ($n = 38$) to 1.09 ($n = 44$)—a descriptive similarity that does not by itself establish exchangeability; the fit treats the draws as approximately exchangeable (Figure 1). We flag one reporting lesson learned the hard way: an interim bookkeeping note quoted $\sigma \approx 0.84$ for the full series by mistake; recomputation shows that value belongs to the late subseries ($\sigma \approx 0.75$ on the final five draws, v29 + v31-HARD), not the campaign. Draws tables make exactly this kind of error detectable.

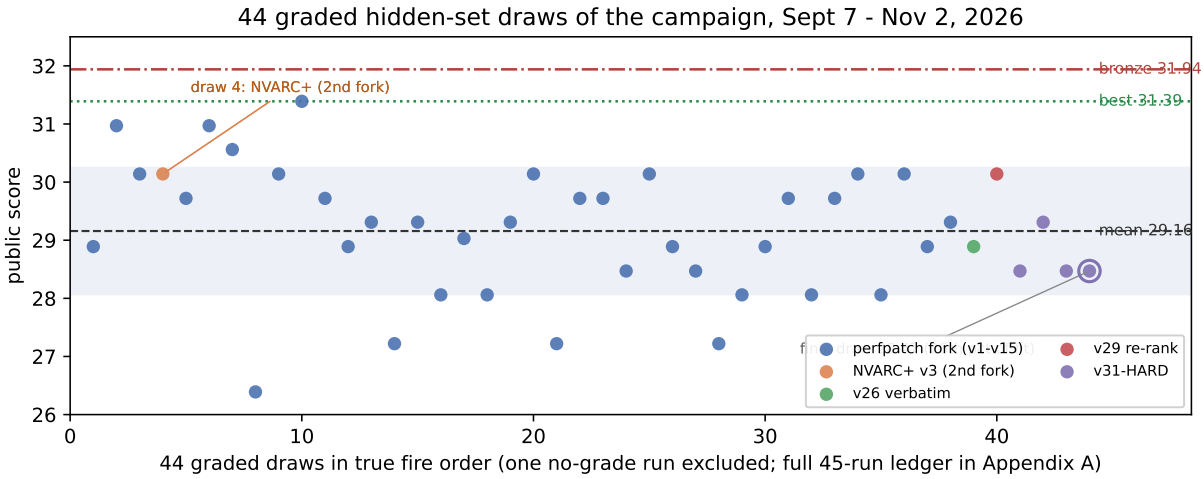


Figure 1: The 44-draw sequence by code-version chapter (draw 4 — the campaign's one draw from a second fork, NVARC+ — marked in orange; final draw annotated). Dashed line: mean; shaded band: $\pm 1\sigma$; dotted green: observed best; dash-dotted red: bronze cutoff (31.94) at the November 2 close, which rose from 31.81 during the campaign; the settled line after late grading was 32.22.

3 Why: leaderboard entries are order statistics

The pipeline's proposal layer is stochastic unless every sampling seed is pinned. With seeds free, each rerun resamples the proposal distribution, and the graded score is one draw from it. This single mechanism explains three community observations at once: (a) forks of the same notebook scatter across a multi-point band while the author sits on top of it; (b) scores drift after semantically neutral edits that perturb RNG consumption order; and (c) "the fork didn't reproduce the author" is usually not skill drift—it is one sample competing against the maximum of many undocumented samples. **A leaderboard of point estimates systematically overstates the reproducibility of everyone on it.** An author's 33.89 may be a skill estimate or an order statistic—the maximum of dozens of private save-and-run cycles that never entered the public record. Without a draws table the two are indistinguishable; with one, fork quality becomes measurable.

4 The free-draw audit: the protocol

We call the procedure the **free-draw audit**. AGI-2's mechanics make it zero-cost: resubmitting an already-published notebook version is a free rerun—it consumes the daily slot but requires no code work, no GPU quota, and no review. For any rerun-scored competition:

1. Fork the strongest reachable public notebook; submit once to establish your band.
2. Pin seeds only when a stable comparison number is needed (accepting that the fixed score may sit below your best draw).
3. Spend every remaining daily slot resampling the *unfixed* version; log each draw with its submission ID.
4. Let leaderboard auto-select keep the maximum; report the full table, not just the max.
5. Evaluate variants only against the draws baseline, never against a single number.

5 What the draws bought

Fitting a normal model to the 44 draws ($\mu = 29.16$, $\sigma = 1.09$) projects quantities that can be checked against what actually happened (Figure 2):

Table 3: Fitted projections vs. observations at the campaign's close (November 2, 2026; all draws graded).

Quantity	Fitted	Observed
Maximum over 44 draws	31.55	31.39 (gap 0.16)
Crossings of bronze 31.94 in 44 draws	0.23 expected	0
Single-draw $P(\geq 31.94)$	0.53%	—
Final graded draw (#44, 24–31 h queue)	28.47 at 26th percentile of fit	28.47 observed (out-of-sample)

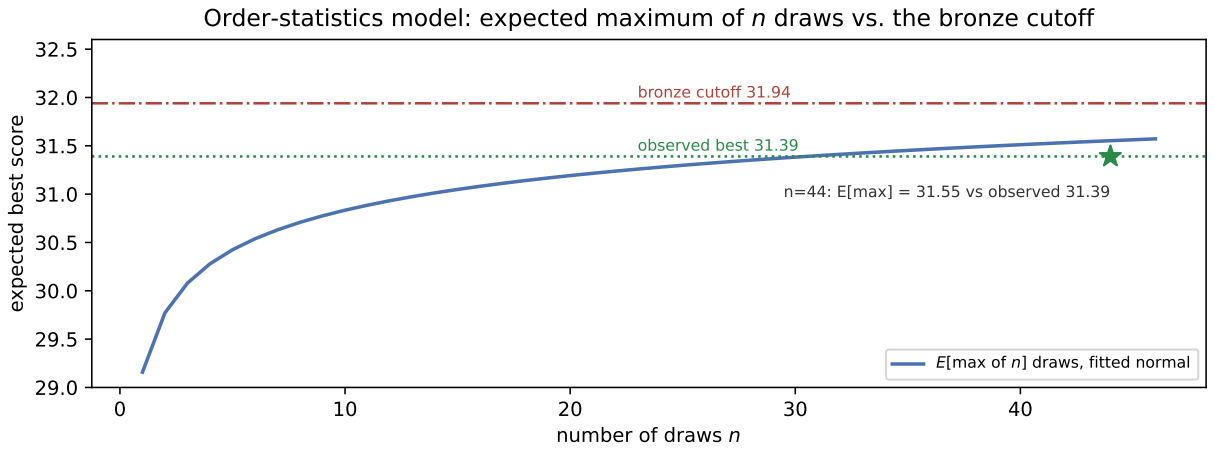


Figure 2: The order-statistics model in action: the expected maximum $E[\max]$ of the fitted distribution as draws accumulate, against the observed best (31.39) and the bronze cutoff (31.94) at the close (settled line: 32.22). At $n = 44$ the fit projects 31.55—a 0.16 gap to the observed maximum.

Two distinct observables—the realized maximum and the realized crossing count—land where the fitted distribution says they should. The agreement is the strongest descriptive check we can offer: related summaries of the same draws, compatible with the approximately exchangeable model, not independent validation of it. It converts the bronze cutoff into a priced lottery: at 0.53% per draw, a team at our mean would need roughly 130 draws for a coin-flip chance at bronze. The comparison that matters is points per draw versus points per engineering day. A draw costs zero code and has strictly non-negative effect under auto-select; engineering has high cost and uncertain sign—our 217-round hypothesis-lab cycle produced only rejections, and the selection battery of Section 6 was adopted only after a held-out bake-off. The experiment ran to completion: we kept one free draw per day until the November 2 deadline, and the final draw (28.47) landed on the 26th percentile of the fitted distribution—an out-of-sample confirmation: the $n = 43$ version of this fit was already fixed in this paper's October 31 draft, before that draw was fired.

6 Selection is not the bottleneck

The draws audit measures variance around a fixed ceiling; the natural question is what moves the ceiling. We ran a two-stage selection study on held-out evaluation tasks, using real candidate pools produced by our test-time-training and search layers.

Stage 1: selector bake-off. We replaced the production selection rule (a product-of-experts-style agreement-minus-NLL score) with a battery of ten scoring algorithms, and evaluated them offline on 37 evaluation tasks using the official top-2 metric. The winner, a quality-weighted agreement vote (*soft agreement*), scored 9.17 against the production default's 7.50, a gain of +1.67 points (+22% relative), concentrated in tasks where calibration disagreed with vote counts. We adopted it as the new default (v29) only after the bake-off; a subsequent daily draw confirmed no regression.

Stage 2: executive verifier. Inspired by evolutionary program-synthesis systems that execute candidates on the demonstration pairs and score them [10, 11], we built an executive verifier: predict output shapes from the visible pairs (a hard contract that fired on 85.1% of 1,000 training tasks with 99.67% precision, 911/914), reject shape-violating candidates, add a cell-consensus bonus, and force a genuinely diverse second attempt. A calibrated simulation over 500 tasks × 3 seeds, holding pools identical, projected +2.10 to +2.27 points, four times the 0.55-point gap that separated us from the 31.94 line at the November 2 close.

Stage 3: the real-pool benchmark refutes the simulation. We then ran the decisive experiment as a research kernel on 40 real evaluation tasks (every third task of the 120-task evaluation spread), with the real production pools, twelve selectors × two attempt modes, consuming no submission quota:

Table 4: Real-pool benchmark on 40 evaluation tasks (top-2 official metric). Selection saturates; diversity harms.

Measurement	Value	Reading
Reference selector (kgmon, top-2)	9.17	baseline
All top-8 selectors (incl. v29 soft agreement)	9.17	+0.00—selection saturated
Verifier variants	9.17	simulated +2.27 did not transfer
Forced-diverse attempt-2 (all selectors)	7.17	−2.00—diversity is harmful
Shape contract fired (of 55 map tasks)	43	zero outcome flips
Decode-time solved subkeys	118/741 (15.9%)	pool quality is the constraint

The executive verifier added exactly nothing, and forcing the second attempt to be different *cost* two tasks; the near-duplicate attempt-2 that a duplicate audit had flagged as "waste" is in fact near-optimal, because a genuinely diverse second candidate scores worse. The shape contract was constructible for 55 tasks in the benchmark's allowed-task map and fired on 43 of them, yet flipped zero outcomes. One labeling note: the 7.50 above is the pre-bake-off default (kgmon) on the 37-task bake-off set, while the 9.17 baseline in the table is measured on the 40-task real-pool set, where the adopted v29 rule and kgmon tie at the same saturated value—the two numbers answer different questions. Meanwhile only 15.9% of decode-time subkeys were solved at all: when the pool contains no correct program, no selector can recover it. **The joint lesson is causal, not anecdotal:** selection saturates once pools contain a correct candidate, and the binding constraint is candidate-pool quality. Training-pool simulations overstated selector gains by roughly 2.3 points; a single real-pool kernel run exposed this at zero submission cost. Our recommended discipline for any rerun-scored competition: (i) audit variance with a draws table, (ii) gate every selection or verification change with a real-pool benchmark before integrating, and (iii) spend the freed engineering budget on generation breadth—broader synthetic augmentation and deeper search—which is where our post-competition effort now goes, consistent with the pool-quality emphasis of the strongest public solutions [6, 11].

7 Reliability under a deadline

Three operational findings generalize beyond our campaign.

Crash isolation. One verbatim-control submission crashed the whole rerun (Kaggle returned status *complete* with no score and the error string "Your notebook hit an unhandled error while rerunning your code"; ref 57646158, preserved in the public submission export), costing a full day's slot and ≈29 points of expectation. The fix—per-task `try/except` isolation around the solve loop—caps the worst case at one task (≈0.83 points). Five subsequent draws ran crash-free through the deadline window. Near the deadline, grading queues stretched to 24–25 hours of PENDING; that is a delay pattern, not a failure pattern, and should not trigger resubmission reflexes.

Automation needs audits. A scheduled fire job failed silently on October 31: the slot was simply lost. We now reconcile "slot opened → submission fired" every cycle. In a quota-priced competition, a silent scheduler bug is as expensive as a code crash.

Third-party dependency risk. Our ARC-AGI-3 champion agent served a 135 GB quantized model attached as a third-party Kaggle model. Between campaigns the owner made it private (API 403; web 404): the platform silently detached it, and every subsequent rerun died 30–40 minutes into a nine-hour run, three draws in a row. The failure was invisible in the generic error string; only an interactive rerun surfaced the missing-mount error. Rehosting 135 GB was not feasible inside the deadline, so the line ended at its season best of 1.82 (all-time 2.58). The lesson is a supply-chain rule for code competitions: *pin or rehost every third-party dependency you cannot afford to lose*, and treat "works when I run it interactively, dies in rerun" as a missing-mount signature first.

8 Replication on ARC-AGI-3

Nothing here is ARC-AGI-2-specific. We ran the same audit on our ARC-AGI-3 agent line: fifteen verbatim reruns of one harness spread 0.36 to 2.58—a 7.2× relative spread (mean 1.67, σ 0.67). In relative terms the effect is larger than on AGI-2: an agent harness can multiply its own score seven-fold on rerun luck alone. Six further draws on the champion v9 harness (October 3–7) scored between 1.21 and 1.82, with 1.82 the season best; the privatization failure of Section 7 then ended the line. Any evaluation that (i) reruns stochastic code on a hidden set and (ii) selects a team's maximum is an order-statistics marketplace: LLM-benchmark submissions with sampling temperature and code competitions with nondeterministic training fall in scope. The audit—log every draw, fit the spread, price draws against engineering, pin dependencies—applies unchanged.

9 The second candidate, and a negative result

Our DSL solver keeps the submission honest. It is a pure-Python exact-fit chain with no learned component; its public notebook self-scores on the open evaluation split and writes the hidden-set submission file, and its full 30-file dependency closure ships as a public dataset. It scored 0 on the hidden set—reported without spin, because it bounds what a CPU-only, no-LLM exact-fit program covers on ARC-AGI-2, and it motivated every rejection in our 217-round lab cycle.

10 Reproducibility

Every graded score is anchored to a Kaggle submission ID: the full 45-submission ledger (Appendix A) is mirrored as a public CSV (`https://www.kaggle.com/datasets/defiaudit/arc-paper-track-2026`) beside a verbatim Kaggle submissions export (November 5, 2026) carrying the fire-time descriptions, statuses, scores, and the failed run's error string; all 45 rows

reconcile exactly. After the campaign the draw notebook's slug stopped resolving publicly (API 404), which our November 5 owner-side check read as removal; a browser check found it alive but renamed and private, its 32-version history intact (v12 and the verbatim v26 included; best shown result 31.39 at v9; the bake-off merged into it) — so version-level comparison is owner-side possible again, though not public. The public code-level record is threefold: the export's fire-time descriptions, SHA-256-checked byte snapshots of the chapter builds in the same dataset, and the still-public DSL solver notebook and dataset plus the `mikelou1` source notebook (self-labeled 33.89). This PDF is the full paper; the Kaggle writeup condenses it, with identical numbers. The protocol needs no privileged data: any team can replicate the audit with one fork and a week of daily resubmits, and the benchmark stage with one research-kernel run.

11 Limitations

The normal fit is an assumption: 44 draws cannot rule out heavier tails, and the seed-pinning conclusion rests on one agreeing pair ($n = 2$). σ drifted mildly across code chapters (1.14 $\rightarrow$ 1.09 full-series; 0.75 on the final five draws, v29 + v31-HARD), so we treat draws from different chapters as only approximately exchangeable. The benchmark stage covers 40 of 120 evaluation tasks, so its zero-effect verdict has roughly $\pm$0.3-point resolution—enough to exclude the simulated +2.27, not to measure sub-half-point effects. The AGI-3 replication shares the same authors and cadence: an independent competition but not an independent team. Our absolute scores are mid-pack; nothing here claims a modeling breakthrough. The contribution is the measurement instrument.

12 Conclusion

We treated our own submission as a system to be measured, and the measurements changed our decisions. Draws tables exposed rerun variance as the dominant term in our fork-cluster entry and priced the bronze lottery; a held-out bake-off found a real +1.67 selection gain; and a real-pool benchmark refuted a promising +2.27 simulation, relocating the binding constraint from selection to candidate-pool quality at zero submission cost. Reliability engineering—crash isolation, automation audits, dependency pinning—protected the remaining expectation under deadline pressure. None of this required a better model; all of it required only that every score be treated as one draw from a distribution that can be measured, priced, and defended. We believe this discipline—*measure the rerun, bench the selector, pin the dependencies*—is immediately reusable by any team competing on rerun-scored benchmarks, and we release every artifact needed to replicate it.

References

[1] F. Chollet. On the Measure of Intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

[2] F. Chollet, M. Knoop, G. Kamradt, B. Landers, and H. Pinkard. ARC-AGI-2: A New Challenge for Frontier AI Reasoning Systems. *arXiv preprint arXiv:2505.11831*, 2025.

[3] ARC Prize Foundation. ARC Prize 2026: Competition overview, rules, and key dates. https://arcprize.org/competitions/2026, 2026.

[4] ARC Prize Foundation. ARC Prize 2026 Paper Prize: rubric and submission requirements. https://arcprize.org/competitions/2026/paper, 2026.

[5] M. Knoop. ARC Prize 2025 Results & Analysis. https://arcprize.org/blog/arc-prize-2025-results-analysis, 2025.

[6] A. Sorokin and J.-F. Puget. NVARC: winner, ARC Prize 2025 high-score track (24.03% at $0.20/task). Code and paper via https://arcprize.org/competitions/2025, 2025.

[7] A. Jolicoeur-Martineau. Less is More: Recursive Reasoning with Tiny Networks. *arXiv preprint arXiv:2510.04871*, 2025. ARC Prize 2025 Paper Award, 1st place.

[8] J. Pourcel, C. Colas, P. Oudeyer. Self-Improving Language Models for Evolutionary Program Synthesis: A Case Study on ARC-AGI. ARC Prize 2025 Paper Award, 2nd place. https://arcprize.org/competitions/2025, 2025.

[9] I. Liao and A. Gu. ARC-AGI Without Pretraining. ARC Prize 2025 Paper Award, 3rd place. https://arcprize.org/competitions/2025, 2025.

[10] J. Berman. From Parrots to Von Neumanns: How Evolutionary Test-Time Compute Achieved State-of-the-Art on ARC-AGI. ARC Prize 2025 runner-up paper. https://arcprize.org/competitions/2025, 2025.

[11] E. Pang. Efficient Evolutionary Program Synthesis. ARC Prize 2025 runner-up paper. https://arcprize.org/competitions/2025, 2025.

[12] D. Franzen, J. Disselhoff, D. Hartmann. Product of Experts with LLMs: Boosting Performance on ARC Is a Matter of Perspective. ARC Prize 2025 honorable mention. https://arcprize.org/competitions/2025, 2025.

[13] ARC Prize Foundation. ARC Prize 2025 Technical Report, *arXiv preprint arXiv:2601.10904*, 2026.

A Full submission ledger

All 45 submissions of the campaign in true fire order (September 7 – November 1, 2026). Graded draws are numbered 1–44; submission 57646158 returned no grade (whole-run crash) and is excluded from every statistic. Fire-time descriptions carried ad-hoc labels ("draw24", "draw30" and "draw32" were each reused twice; the label "draw44" was skipped after the October 31 slot was silently lost to an automation fault, so no submission carries it) — the public CSV preserves those descriptions verbatim, a verbatim Kaggle submissions export (November 5, 2026) reproduces every row of this table directly from the platform, and the October 31 slot therefore produced no row here.

Table 5: Full submission ledger (canonical graded numbering; fire order top to bottom).

#	Kaggle ref	Fired (UTC)	Chapter	Score
1	56075910	Sep 07	fork v1–v15	28.89
2	56100667	Sep 08	fork v1–v15	30.97
3	56125103	Sep 09	fork v1–v15	30.14
4	56148554	Sep 10	NVARC+ v3	30.14
5	56168010	Sep 11	fork v1–v15	29.72
6	56191452	Sep 12	fork v1–v15	30.97
7	56242479	Sep 15	fork v1–v15	30.56
8	56278055	Sep 16	fork v1–v15	26.39

#	Kaggle ref	Fired (UTC)	Chapter	Score
9	56290322	Sep 17	fork v1–v15	30.14
10	56316858	Sep 18	fork v1–v15	31.39
11	56344456	Sep 19	fork v1–v15	29.72
12	56392133	Sep 20	fork v1–v15	28.89
13	56410652	Sep 21	fork v1–v15	29.31
14	56446773	Sep 22	fork v1–v15	27.22
15	56476966	Sep 23	fork v1–v15	29.31
16	56506943	Sep 24	fork v1–v15	28.06
17	56534453	Sep 25	fork v1–v15	29.03
18	56563942	Sep 26	fork v1–v15	28.06
19	56777145	Oct 02	fork v1–v15	29.31
20	56795558	Oct 03	fork v1–v15	30.14
21	56828541	Oct 04	fork v1–v15	27.22
22	56838789	Oct 05	fork v1–v15	29.72
23	56866534	Oct 06	fork v1–v15	29.72
24	56895967	Oct 07	fork v1–v15	28.47
25	56929499	Oct 08	fork v1–v15	30.14
26	56982331	Oct 09	fork v1–v15	28.89
27	57024478	Oct 10	fork v1–v15	28.47
28	57058105	Oct 11	fork v1–v15	27.22
29	57091249	Oct 12	fork v1–v15	28.06
30	57126165	Oct 13	fork v1–v15	28.89
31	57160155	Oct 14	fork v1–v15	29.72
32	57312203	Oct 17	fork v1–v15	28.06
33	57331947	Oct 18	fork v1–v15	29.72
34	57374759	Oct 19	fork v1–v15	30.14
35	57415205	Oct 20	fork v1–v15	28.06
36	57456865	Oct 21	fork v1–v15	30.14
37	57543492	Oct 23	fork v1–v15	28.89
38	57576234	Oct 24	fork v1–v15	29.31
39	57641015	Oct 25	v26	28.89
—	57646158	Oct 26	v26	—
40	57682428	Oct 27	v29	30.14
41	57717109	Oct 28	v31-HARD	28.47
42	57752643	Oct 29	v31-HARD	29.31
43	57786357	Oct 30	v31-HARD	28.47
44	57855664	Nov 01	v31-HARD	28.47

Data availability. Every row above was reconciled against the verbatim submissions export of November 5, 2026 (fire-time descriptions, statuses, scores, and the failed run's error string), shipped next to the CSV at `https://www.kaggle.com/datasets/defiaudit/arc-paper-track-2026`; all 45 match exactly. The draw fork's slug stopped resolving publicly after the campaign (API 404); the bake-off survives merged into the same kernel; the owner-side check of November 5, 2026 is inventoried in Section 10. The chapter column rests on the export's fire-time descriptions (39 of the 44 graded draws) plus, for the other five, ledger notes and the dataset's SHA-256-checked byte snapshots. Author interpretations: the no-grade cause is read from Kaggle's error string; draw 3's seed patch is a ledger note (no fire description for ref 56125103 in the export); the source notebook's 33.89 is a slug-and-title self-label (page now shows 32.22; no independent fork-time record); it remains public (69.72 final, rank 33 at our November 5 pull).